NATIONAL UNIVERSITY OF OSTROH ACADEMY
Department of Information Technologies and Data Analytics

Lab 9.
Creating interactive DOM and event handling: event delegation, forms, and dynamic UI accessibility

Modern Web Development: HTML, CSS, JS/TS

Deadline: 19.10.2026

1. Aim

Learn to build an interactive user interface with native DOM APIs without relying on frontend frameworks: instantiating elements from templates, handling user interaction via event delegation, working with forms and modal dialogs, and managing focus so the application is fully operable with a keyboard and screen reader. Learn to audit AI-generated JavaScript code against specifications and real browser behavior, rather than assuming it works because it "looks functional."

Expected outcome: the "My Watchlist" application, where every component was first drafted by an AI agent, then audited against the assignment and fixed by hand; a clean "Passed: 33 of 33" in the self-check suite, and a Git history where each AI draft commit is followed by your corresponding fix commit.

2. Objectives and learning outcomes

After completing this lab, you must be able to:

- Locate elements using `querySelector*`, distinguish between live and static collections, and handle `dataset` values as strings.
- Create elements from `<template>`, insert text and attributes safely using `textContent`, and explain why interpolating data into `innerHTML` causes XSS.
- Explain capturing and bubbling phases, the distinction between `target` and `currentTarget`, the meaning of `this` in event handlers, and how to use `closest()`.

- Implement event delegation and explain why delegated handlers are bound to a container node rather than a selector.
- Handle forms properly: `submit` events and `preventDefault()`, Enter-key implicit submission, `input` vs. `change`, `reset`, `valueAsNumber`, and the Constraint Validation API.
- Manage focus deliberately: `focus()`, `tabindex`, preserving focus after element removal, and modal `<dialog>` interactions.
- Communicate dynamic state to assistive technologies: `aria-pressed`, `aria-invalid`, `aria-describedby`, and live regions (`role="status"`).
- Audit AI-generated code against the assignment, the self-check, and DevTools, recording the audit trail in Git.

3. Lab tasks

1. **At home, before class:** work through the research tasks (§4) and conduct the experiments: for each one, analyze the code and write down your expected result **before** running it in the browser.
2. **In class:** download the My Watchlist starter project, commit it to Git unchanged, and confirm that the self-check initially reports “Passed: 0 of 33” (§5.1).
3. **In class:** complete Challenges 1–4 (§5.3–5.6): for each challenge, have the AI agent write a draft, commit it, audit it against the assignment, and fix all problems by hand.
4. **At home:** go through the assignment's manual checklist on your final build, deploy the project to GitHub Pages or Vercel, prepare your report (§6), and defend your work (§7).

4. Research tasks

Before starting the practical part, study the sources and conduct the experiments. General reference — Introduction to the DOM — MDN; accessibility requirements — WCAG 2.2 and the ARIA Authoring Practices Guide; browser behavior is governed by the HTML Living Standard.

These experiments explore browser behavior rather than application logic: you do not need to write fixes for them. The “Where this occurs” note in each experiment highlights where this behavior turns into a defect in the “My Watchlist” application if overlooked.

Experiment page. The starter project (watchlist-starter.zip, the same as in §5) includes `experiments.html`. It contains 15 numbered blocks — one for each experiment. The block number and title match the experiment number and title below. Each block is a `<section id="exp-N">` element, and the code snippets query their elements within that block: `document.querySelector('#exp-1 …')`. Because the HTML for each block is displayed in the description, you can analyze the code without opening the page.

Extract the archive and start Live Preview for `experiments.html`. Open the page in a browser at the `http://127.0.0.1:…` address reported by Live Preview (rather than the built-in VS Code tab), and open DevTools (F12, or Cmd+Option+I on macOS) → Console.

Use Chrome or Edge: Live Expressions and Preserve log are present in their DevTools, and click/Tab events in Safari and Firefox on macOS handle focus differently (assignment, §6).

Experiment protocol:

1. Read the description, **analyze** the snippet, and write down your predicted result — the answer to the "Analyze" prompt.
2. Reload `experiments.html` and paste the snippet into the Console as a whole. Reload before each experiment: snippets mutate the DOM.
3. If any predicted result differed from the actual result, explain where your analysis was mistaken.

The questions in the "Where this occurs" sections are for oral defense preparation and do not need written answers. Compile your results into a table — it is part of your report:

#	What was investigated	My analysis	Actual result	Explanation (if mismatched)
1	Live and static collections	...	...	...

4.1. DOM API

Study:

- Locating DOM elements using selectors — MDN
- HTMLCollection and NodeList — MDN
- Node: textContent and Element: innerHTML, Security considerations section — MDN
- <template>, Element: replaceChildren(), HTMLElement: dataset — MDN

Investigate: what `querySelector` and `querySelectorAll` return when no match is found; live vs. static collections; differences between `textContent`, `innerHTML`, and `innerText`; why `<template>` content is inert; what `cloneNode(true)` does.

Experiments:

1. **Live and static collections.** Block `#exp-1` on `experiments.html`:

```
<section id="exp-1">
  <h2>1 · Live and static collections</h2>
  <ul>
    <li class="item">First</li>
    <li class="item">Second</li>
    <li class="item">Third</li>
    <li class="item">Fourth</li>
    <li class="item">Fifth</li>
    <li class="item">Sixth</li>
  </ul>
</section>
```

   The snippet retrieves these items using `getElementsByClassName` and loops to remove the `item` class from each. The last line counts how many elements with that class

remain.

```
const items = document.getElementsByClassName('item');
for (let i = 0; i < items.length; i++)
items[i].classList.remove('item');
document.querySelectorAll('.item').length;
```

Analyze: what number does the last line return? Then reload the page, replace the first line with `const items = document.querySelectorAll('.item');`, and analyze again.
Where this occurs: clearing `#results` before rendering fresh cards (Challenge 1): a loop deleting cards one by one traverses the live collection `list.children`. Explain how these collections differ, which one is safe to iterate while mutating the DOM, and what `replaceChildren()` does instead.

2. **Inserting HTML strings four different ways.** Block `#exp-2` on `experiments.html`:

```
<section id="exp-2">
  <h2>2 · Inserting HTML strings four different ways</h2>
  <p>textContent: <output class="as-text"></output></p>
  <p>innerHTML: <output class="as-html"></output></p>
</section>
```

`payload` constructs a string with an `<img>` tag: the image does not exist, and its `onerror` handler logs how the string reached the page — but only if the browser parsed the string into an `<img>` element and attempted to load it. The snippet sets the string via `textContent` in the first `<output>`, via `innerHTML` in the second, as well as via `innerHTML` of an unattached `<div>`, and via `innerHTML` of a `<template>` element.

```
const payload = (via) => `<img src="x" onerror="console.log('onerror:
${via}')">`;
document.querySelector('#exp-2 .as-text').textContent =
payload('textContent');
document.querySelector('#exp-2 .as-html').innerHTML =
payload('innerHTML');
document.createElement('div').innerHTML = payload('div');
document.createElement('template').innerHTML = payload('template');
```

Analyze: which of the four techniques log a message to the Console? What will be visible in section 2 of the page?
Where this occurs: show titles or notes containing HTML markup (Challenges 1 and 4), and show summaries from TVmaze data that contain HTML (Challenge 4). Explain which methods are data-safe and why stripping HTML tags via a temporary `<div>` is unsafe.

3. **Values from `dataset`.** Block `#exp-3` on `experiments.html`:

```
<section id="exp-3">
  <h2>3 · Values from dataset</h2>
  <ul>
    <li data-id="82">Game of Thrones</li>
  </ul>
</section>
```

Each card in the application stores its show id in the same way. The snippet reads `data-id` and compares it to the number 82.

```
const li = document.querySelector('#exp-3 li');
[li.dataset.id, typeof li.dataset.id, li.dataset.id === 82,
Number(li.dataset.id) === 82];
```

Analyze: which four values will the array contain?
Where this occurs: the id read from a card is passed into functions in `src/state.js` (Challenge 2). Explain why equality comparison with a number fails and how to obtain a number correctly.

4. **`innerHTML +=` and event handlers.** Block `#exp-4` on `experiments.html`:

```
<section id="exp-4">
  <h2>4 · innerHTML += and event handlers</h2>
  <button type="button">Button with handler</button>
</section>
```

The snippet attaches a `click` listener to the button, clicks it, appends a new paragraph using `innerHTML +=`, and clicks the button again.

```
const box = document.querySelector('#exp-4');
box.querySelector('button').addEventListener('click', () =>
console.log('click'));
box.querySelector('button').click();
box.innerHTML += '<p>New line</p>';
box.querySelector('button').click();
```

Analyze: how many times does the Console output "click"?
Where this occurs: appending new cards to a list via string concatenation (Challenges 1 and 2). Explain what `innerHTML +=` does to elements that already exist inside the container.

4.2. Events and event delegation

Study:

- Introduction to events and Event bubbling, Event delegation section — MDN
- EventTarget: addEventListener() — MDN, especially sections on `this` and duplicate listener registration

- Event: target, Event: currentTarget, Element: closest() — MDN
- Button pattern — ARIA APG

Investigate: three phases of event propagation; `target` vs. `currentTarget`; why `this` in a regular function listener references the element, whereas in an arrow function it does not; what `closest()` returns when no matching ancestor exists; what happens when registering the same handler twice; why event delegation works for dynamically added elements; built-in behaviors of `<button>` compared to a `<div>` with a `click` handler; what `isTrusted` indicates.

Experiments:

5. **Event propagation phases.** Block `#exp-5` on `experiments.html`:

```
<section id="exp-5">
  <h2>5 · Event propagation phases</h2>
  <ul>
    <li><button type="button">Click me</button></li>
  </ul>
</section>
```

   The snippet registers capturing and bubbling `click` listeners on `<ul>`, `<li>`, and the button. Click the button with the mouse after running.

```
const ul = document.querySelector('#exp-5 ul');
for (const el of [ul, ul.querySelector('li'),
ul.querySelector('button')]) {
  const name = el.tagName.toLowerCase();
  el.addEventListener('click', () => console.log(name, 'capture'), {
capture: true });
  el.addEventListener('click', () => console.log(name, 'bubble'));
}
```

   Analyze: in what sequence will the six lines be logged?
   Where this occurs: a delegated handler bound to `#results` (Challenge 2) receives button clicks due to these phases. Explain the sequence using the three propagation phases.

6. **`target`, `currentTarget`, and `this`.** Block `#exp-6` on `experiments.html`:

```
<section id="exp-6">
  <h2>6 · target, currentTarget, and this</h2>
  <button type="button">
    <svg viewBox="0 0 24 24" aria-hidden="true" focusable="false">
      <path d="…" fill="currentColor" />
    </svg>
    Button with icon
  </button>
</section>
```

The <svg> is a star icon; <path> is its outline. The snippet attaches a regular function handler and an arrow function handler to the button, plus a delegated handler to the block container. Click the filled part of the star, then the text of the button, then the section heading.

```
const button = document.querySelector('#exp-6 button');
button.addEventListener('click', function (event) {
  console.log('function:', event.target.tagName,
event.currentTarget.tagName, this.tagName);
});
button.addEventListener('click', (event) => {
  console.log('arrow:', event.target.tagName, event.currentTarget ===
button, this === window);
});
document.querySelector('#exp-6').addEventListener('click', (event) => {
  console.log('closest:', event.target.closest('button'));
});
```

Analyze: what does each line output for each of the three clicks?
Where this occurs: the "Add to list" button contains an SVG icon (Challenge 2). Explain what determines `event.target` and what `closest()` returns when no button is among the clicked node's ancestors.

7. **The same listener attached twice.** Block `#exp-7` on `experiments.html`:

```
<section id="exp-7">
  <h2>7 · The same listener attached twice</h2>
  <button type="button">Counter</button>
</section>
```

The snippet attaches four listeners that increment `count`: two arrow functions that look the same, and the same function `named` twice. Then it clicks the button once.

```
const button = document.querySelector('#exp-7 button');
let count = 0;
const named = () => count++;
button.addEventListener('click', () => count++);
button.addEventListener('click', () => count++);
button.addEventListener('click', named);
button.addEventListener('click', named);
button.click();
count;
```

Analyze: what will `count` equal?

Where this occurs: attaching event listeners inside a function that re-renders the list (Challenge 2). Explain when the browser registers the same listener again and when it does not.

8. **`<div>` vs. `<button>`.** Block `#exp-8` on `experiments.html`:

```
<section id="exp-8">
  <h2>8 · div vs button</h2>
  <div class="fake">Div with handler</div>
  <button type="button">Real button</button>
</section>
```

The `fake` class merely styles the `<div>` to resemble a button. The snippet adds `click` listeners logging `isTrusted`. After running:

- click both elements with the mouse;
- click the "Counter" button in `#exp-7` and press Tab repeatedly;
- when focused on "Real button", press Enter, then Space;
- run in Console `div.click(); button.click();`.

```
const div = document.querySelector('#exp-8 .fake');
const button = document.querySelector('#exp-8 button');
div.addEventListener('click', (event) => console.log('div',
event.isTrusted));
button.addEventListener('click', (event) => console.log('button',
event.isTrusted));
```

Analyze: what does the Console display after each action? Does the `<div>` receive focus using the Tab key?
Where this occurs: buttons on cards and in dialogs (Challenges 1, 2, and 4). Explain what `<button>` provides natively without JavaScript and what `isTrusted` indicates.

4.3. Forms

Study:

- Client-side form validation — MDN
- Constraint validation API and HTMLInputElement: setCustomValidity() — MDN
- HTMLFormElement: submit event, reset event, requestSubmit() — MDN
- Element: input event and change event — MDN
- Implicit submission — HTML Living Standard

Investigate: when pressing Enter submits a form and what happens without `preventDefault()`; differences between `submit` and button `click`; when `input` vs. `change` fire; when `reset` occurs relative to clearing field values; `value` vs. `valueAsNumber`; what `validity`, `checkValidity()`, and `setCustomValidity()` do; what the `novalidate` attribute controls.

Experiments:

9. **Enter in a form.** Block `#exp-9` on `experiments.html`:

```
<section id="exp-9">
  <h2>9 · Enter in a form</h2>
  <form>
    <label>Search <input name="q" autocomplete="off" /></label>
    <button>Find</button>
  </form>
</section>
```

   The snippet attaches a `click` listener to the button and a `submit` listener to the form. First enable **Preserve log** in the Console so logs do not vanish on page navigation. Then focus the input, type `drama`, and press Enter.

```
const form = document.querySelector('#exp-9 form');
form.querySelector('button').addEventListener('click', (event) =>
console.log('click', event.isTrusted));
form.addEventListener('submit', () => console.log('submit'));
```

   Analyze: which logs appear in the Console? What happens to the page and the browser address bar?
   Where this occurs: the search input in the filter form (Challenge 3) and the note form (Challenge 4). Explain what triggers implicit form submission and how to prevent page reload.

10. **`value` and `valueAsNumber`.** Block `#exp-10` on `experiments.html`:

```
<section id="exp-10">
  <h2>10 · value and valueAsNumber</h2>
  <label>Number <input type="number" /></label>
</section>
```

    The snippet assigns 7 to the input and adds 1 in two ways, then clears the input and repeats.

```
const input = document.querySelector('#exp-10 input');
input.value = '7';
const filled = [input.value + 1, input.valueAsNumber + 1];
input.value = '';
[filled, [input.value + 1, input.valueAsNumber + 1]];
```

    Analyze: which two pairs of values does the snippet return?
    Where this occurs: the "My rating" field (Challenge 4), where `saveNote()` requires a number. Explain the data types returned and why one evaluates to `NaN` when the input is empty.

11. **`setCustomValidity`.** Block `#exp-11` on `experiments.html`:

```
<section id="exp-11">
  <h2>11 · setCustomValidity</h2>
  <label>Rating from 1 to 10 <input type="number" min="1" max="10" />
</label>
</section>
```

The snippet sets the value to 42, sets a custom validation message, then updates the value to 5 and inspects validity.

```
const input = document.querySelector('#exp-11 input');
input.value = '42';
if (input.valueAsNumber > 10) input.setCustomValidity('No more than
10');
input.value = '5';
[input.validity.valid, input.validity.customError,
input.checkValidity(), input.validationMessage];
```

Analyze: which four values will the array contain?
Where this occurs: validating rating inputs in the note form (Challenge 4). Explain when custom validation messages set with `setCustomValidity` are cleared.

12. **The `reset` event.** Block `#exp-12` on `experiments.html`:

```
<section id="exp-12">
  <h2>12 · The reset event</h2>
  <form>
    <label>Filter <input name="q" /></label>
    <button type="reset">Reset</button>
  </form>
</section>
```

The snippet adds a `reset` handler that logs the current field value, sets the input to `drama`, and triggers `.reset()`.

```
const form = document.querySelector('#exp-12 form');
form.addEventListener('reset', () => console.log('in reset:',
JSON.stringify(form.elements.q.value)));
form.elements.q.value = 'drama';
form.reset();
console.log('after reset():', JSON.stringify(form.elements.q.value));
```

Analyze: which two lines will the Console log? Then type something into the field and click the "Reset" button: what does the handler log this time?
Where this occurs: the "Reset" button in the filter form (Challenge 3). Explain when the `reset` event fires relative to when input values are cleared.

4.4. Focus, dialogs, and accessibility

Study:

- HTMLElement: focus() and tabindex — MDN
- <dialog> and HTMLElement: inert — MDN; Dialog (Modal) pattern — ARIA APG
- ARIA live regions, aria-pressed, aria-invalid, aria-describedby — MDN
- WCAG 2.2: 2.1.1 Keyboard, 2.4.3 Focus Order, 2.4.7 Focus Visible, 2.5.3 Label in Name, 3.3.1 Error Identification, 4.1.2 Name, Role, Value, 4.1.3 Status Messages

Investigate: which elements are natively focusable without `tabindex`; `tabindex="-1"` vs. `tabindex="0"`; where focus moves when a currently focused element is removed; how `showModal()` affects the rest of the page and where `close()` returns focus; what an accessible name is and how browsers compute it; why a live region must already be present in the DOM before its text content changes. You can hear the announcements only with a screen reader (Narrator, NVDA, VoiceOver); this is optional.

For experiments 13–15, create a **Live Expression** in the Console for `document.activeElement`: it tracks which element currently has focus.

Experiments:

13. **Removing a focused element.** Block `#exp-13` on `experiments.html`:

    ```
    <section id="exp-13">
      <h2>13 · Removing a focused element</h2>
      <button type="button">I will be removed</button>
    </section>
    ```

    The snippet focuses the button, deletes it from the DOM, and inspects where focus lands.

    ```
    const button = document.querySelector('#exp-13 button');
    button.focus();
    const before = document.activeElement === button;
    button.remove();
    [before, document.activeElement];
    ```

    Analyze: which two values will the array contain?
    Where this occurs: the list is re-rendered while focus was on a button of an old card (Challenges 2, 3, and 4). Explain where focus goes and what that means for a keyboard user.

14. **`focus()` and `tabindex`.** Block `#exp-14` on `experiments.html`:

    ```
    <section id="exp-14">
      <h2>14 · focus() and tabindex</h2>
      <div>Plain div</div>
    </section>
    ```

The snippet calls `focus()` on the `<div>`, adds `tabindex="-1"`, and calls `focus()` again.

```
const box = document.querySelector('#exp-14 div');
box.focus();
const plain = document.activeElement === box;
box.tabIndex = -1;
box.focus();
[plain, document.activeElement === box];
```

Analyze: which two values will the array contain? Then click the button in `#exp-13` and press Tab: does focus land on this `<div>`?
Where this occurs: when no cards remain after filtering, focus must move to the list heading (Challenge 3). Explain the difference between `tabindex="-1"` and `tabindex="0"`.

15. **Modal dialog and focus.** Block `#exp-15` on `experiments.html`:

```
<section id="exp-15">
  <h2>15 · Modal dialog and focus</h2>
  <button type="button" class="opener">Open dialog</button>
  <button type="button" class="outside">Button outside dialog</button>
  <dialog>
    <p>Modal dialog</p>
    <button type="button">Button inside dialog</button>
  </dialog>
</section>
```

The snippet focuses "Open dialog", opens the dialog via `showModal()`, attempts to move focus outside, and then closes the dialog.

```
const section = document.querySelector('#exp-15');
const dialog = section.querySelector('dialog');
const opener = section.querySelector('.opener');
const outside = section.querySelector('.outside');
opener.focus();
dialog.showModal();
outside.focus();
console.log('after outside.focus():',
document.activeElement.textContent.trim());
dialog.close();
console.log('after close():',
document.activeElement.textContent.trim());
```

Analyze: what will both lines output?
Then reload the page and run the second variation: here the opening button is removed while the dialog is open.

```
const section = document.querySelector('#exp-15');
const dialog = section.querySelector('dialog');
const opener = section.querySelector('.opener');
opener.focus();
dialog.showModal();
opener.remove();
dialog.close();
```

Analyze: where does focus land after `close()`? Verify with Live Expressions.
Where this occurs: the "Details" dialog and saving a note, after which the card is re-rendered (Challenge 4). Explain what `showModal()` does to the rest of the page and when the browser restores focus on its own.

4.5. AI-generated DOM code

Investigate common default patterns AI agents use in DOM code unless explicitly told otherwise:

- building markup via `innerHTML` and template literals with raw data;
- loading `data/shows.json` via `fetch`, bypassing `src/state.js`;
- adding event listeners to each card individually — either once on load or repeatedly during every re-render;
- reading `event.target` without `closest()`;
- forgetting to cancel implicit form submission;
- recreating the status live region element or omitting it altogether;
- inserting summaries via `innerHTML` or attempting to sanitize HTML through a temporary `<div>`;
- constructing modal popups using plain `<div>` elements;
- displaying validation errors using `alert()`;
- dropping focus after re-rendering cards;
- introducing unnecessary timers: debounce for search, or `setTimeout` / `requestAnimationFrame` before `focus()`;
- adding redundant or harmful ARIA: `role="button"` on `<div>` without keyboard support, or `aria-label` overriding visible text (WCAG 2.5.3).

These are the exact findings to look for in the AI drafts during §5.

5. Practical part: AI draft → audit → hand-crafted fixes

The **“My Watchlist”** starter project is a catalog of 30 shows from TVmaze (the same dataset as in Lab 8). In the completed application, users filter shows by title and genre, toggle shows with the "Add to list" button, view the "In list: N" counter, open descriptions via "Details", and save their own rating with a personal note. The HTML, CSS, sample data, and state management module `src/state.js` are already implemented. Only the JavaScript layer that renders cards and handles user interaction is empty, meaning the initial page displays the filter form and an empty list.

Your task is to bring the application to full functionality in four steps (challenges). Each challenge targets a specific part of the application through the same workflow (§5.2): the AI agent writes a draft from a short prompt, you commit it to Git, audit it against the assignment, and fix all problems by hand. Complete the challenges in sequence: each builds on the previous ones.

How to verify. Exact rules for each challenge are in the assignment, §4. They are numbered: C4.2 refers to rule 2 of Challenge 4. The `check.html` page tests these rules in the browser and displays them by challenge block: “C1 · Cards”, “C2 · Card buttons”, etc. Each rule in a block uses the same number as in the assignment: ✓ means passing, ✗ means failing, accompanied by an explanation. Run it as often as you like: it does not alter project files. What `check.html` cannot verify (real keyboard interaction and visible focus outlines) is covered by the assignment's manual checklist, §6.

5.1. Setup

1. Download watchlist-starter.zip and extract it into a **separate repository** or into a `lab-9/` folder in your shared coursework repository. Make your first commit **unchanged**.
2. Open `index.html` via Live Preview in Chrome or Edge, as in §4: you should see the filter form, an empty list, and a clean Console with no errors.
3. Open `check.html` the same way: it should report “Passed: 0 of 33”. Review what the automated check runner does not cover.
4. `check.html`, `check/`, `data/`, and `src/state.js` must remain unchanged in every commit.

5.2. Rules for working with an AI agent

- **Research tasks (§4).** Write “My analysis” yourself, without AI assistance and prior to running code snippets.
- **Challenges.** Complete each challenge in four steps. The examples below illustrate Challenge 2.
  1. **AI draft.** Set your IDE AI agent to Agent mode (where it can directly edit files). Close editor tabs for `check.html` and files in `check/`: many agents automatically include open files in their context. Paste **only** the short prompt provided in the challenge section, word for word. Do not give the agent the assignment, `check.html`, or clarifications of your own: the goal is to observe what the agent produces by default.
  2. **Draft commit.** Without fixing anything, commit everything modified by the agent with the commit message `Challenge 2: AI draft`. Commit even if the draft is broken: this commit serves as the baseline against which your fixes are evaluated. The only exception is the set of protected files (§5.1). If the agent modified them, revert them before committing using `git restore check.html check data src/state.js`, remove any new files in those directories, and document this in your report.
  3. **Audit.** Open `check.html` and navigate to the challenge block. For every ✗, identify which rule of the assignment was broken and why. Review the Git diff of the draft against the assignment's constraints, §5 (`innerHTML` with data, timers, `fetch`): `check.html` does not detect all of them. Then go through the lines of the assignment's manual checklist tagged with this challenge's number.
  4. **Hand-crafted fixes.** The AI may explain a check, a rule, or an error, but **must not write** the fix code. Once all checks in this block and all preceding blocks show ✓, commit your fixes in a single commit: `Challenge 2: fixes`. If the agent modified code from earlier challenges, fix any regressions there as well.

  If a draft passes its entire block on the first try without breaking earlier challenges, omit the fixes commit and document how you verified it in your report summary.
  Agents often implement more than requested: for example, drafting the Challenge 4 dialog already during Challenge 2. Keep that code in the draft commit, but audit and fix it when you reach the corresponding challenge. Still give the agent the exact prompt for each challenge when you arrive at it.

Example prompt for asking explanations (in non-editing mode — ask / plan):

> Explain why this check is failing. Below are its name, message, and my code. Do not write the fixed code: identify the DOM mechanism or accessibility rule I likely overlooked, and explain how to verify it in DevTools.

In your report, state the name and version of the agent, model, and date of work.

5.3. Challenge 1 — cards

Expected outcome. After page load, the “Shows” list displays 30 cards, one for each show in `data/shows.json`, in the original order. Each card displays the title, year, genres, TVmaze rating, and two buttons: “Add to list” and “Details”. Above the list, the status line announces how many shows were found. The buttons do not need to be functional yet: that is Challenge 2.

What is provided. The card template `<template id="show-card">` in `index.html` with markup and data slots (`data-field`), the empty `#results` list, and the `#status` live region. On initialization, `main.js` calls `renderShows()` and passes the shows array to display.

Code: `renderShows(list)` in `src/render.js`.

Prompt for AI agent:

> Render the shows from data/shows.json in the #results list: title, year, genres, rating, "Add to list" and "Details" buttons.

Verification: open `check.html`: all checks in “C1 · Cards” must show ✓. Experiments 1, 2, 4, and 8 are relevant here.

5.4. Challenge 2 — card buttons

Expected outcome. “Add to list” adds a show to your watchlist, and a second click removes it. The button reflects state (the icon fills), and the "In list: N" counter in the header updates. “Details” opens the show description: the modal dialog itself is built in Challenge 4; here it is sufficient to call `openDetails(id)`. Buttons function identically via mouse and keyboard, as well as after every list re-render. After clicking “Add to list”, focus remains on that button, even if the card was re-rendered.

What is provided. `toggle(id)` and `watchlist` in `src/state.js`, the `#watchlist-count` element in the header, styles for `[aria-pressed="true"]`, and an empty stub for `openDetails(id)` in `src/details.js`.

Code: `initList(refresh)` in `src/list.js`. `main.js` calls it once after the first list render; `refresh()` re-renders the list.

Prompt for AI agent:

> Make "Add to list" add a show to my watchlist, and "Details" open the description.

Verification: open `check.html`: all checks in “C2 · Card buttons” must show ✓. Then complete the C2 rows in the manual checklist (assignment, §6). Experiments 3–8 and 13 are relevant.

5.5. Challenge 3 — filters

Expected outcome. The form above the list filters cards: searching by title filters on input, while genre and "Only my watchlist" filter immediately on change; all active filters combine with logical AND. The status line updates with each change. Pressing Enter in the search input does not reload the page, and "Reset" restores all 30 shows. When removing a show while "Only my watchlist" is active, its card disappears and focus moves to an adjacent card.

What is provided. The `#filters` form with `query`, `genre`, and `mine` fields and a "Reset" button. `visibleShows({ query, genre, mine })` in `src/state.js` handles filtering logic, and `main.js` passes whatever `readFilters()` returns. Your task is to read form values and trigger a re-render on changes.

Code: `readFilters()` and `initFilters(refresh)` in `src/filters.js`; focus management after card removal in `src/list.js`.

Prompt for AI agent:

> Add filtering by title and genre and an "only my watchlist" toggle.

Verification: open `check.html`: all checks in “C3 · Filters” must show ✓. Then complete the C3 rows in the manual checklist (assignment, §6). Experiments 9, 12, 13, and 14 are relevant.

5.6. Challenge 4 — "Details" dialog and note form

Expected outcome. “Details” opens a modal window with the show title, description, and a link to the TVmaze page. While open, the rest of the page is inert. Esc or "Cancel" closes the dialog, returning focus to the button that opened it. Descriptions in the data contain TVmaze HTML markup, but must be rendered as plain text.

Inside the dialog, below the description, is a form: “My rating” (mandatory integer from 1 to 10) and “Note” (optional). The form rejects invalid ratings: an error message appears next to the field, and the dialog remains open without `alert()`. Upon saving, the dialog closes, the card displays "My rating: 8/10" and the note, the status line announces the save, and focus returns to the “Details” button on that card. Reopening the dialog displays the saved data for that specific show.

What is provided. `index.html` contains `<dialog id="details">` with `#details-title`, `#details-summary`, `#details-link`, and `#note-form` (fields `rating` and `note`, error container `#note-rating-error`, "Save" and "Cancel" buttons). JavaScript only fills in and opens the dialog. `src/state.js` provides `getShow(id)`, `saveNote()`, and `notes`, while the card template includes slots for rating and note.

Code: `openDetails(id)` and `initDetails(refresh)` in `src/details.js`. The “Details” button (Challenge 2) calls `openDetails`, which populates the dialog and form. `initDetails`

wires up dialog close buttons and form submission. Card rating rendering is handled in `src/render.js`.

Prompt for AI agent:

> Make a "Details" dialog with the show description and a validated "my rating and note" form.

Verification: open `check.html`: all checks in “C4 · Details dialog and note” must show ✓. Then complete the C4 rows in the manual checklist (assignment, §6). Experiments 2, 8–11, 13, and 15 are relevant.

5.7. Commits

For each challenge, maintain two commits in Git:

1. `Challenge N: AI draft` — the code generated by the agent, completely unmodified;
2. `Challenge N: fixes` — all your hand-crafted corrections to that draft.

If a draft passed its block on the first attempt without breaking earlier challenges, omit the fixes commit (§5.2).

Viewing `Challenge N: fixes` on GitHub shows the exact diff between the AI draft and your corrections. During defense, you will use this diff to explain what was missing or incorrect in the AI draft (§7).

After completing Challenge 4, go through the assignment's full manual checklist once more on your final build and publish the project to GitHub Pages or Vercel (instructions in the starter project `README.md`).

6. Report requirements

Email subject:

```
[Web] Lab 9: Creating interactive DOM and event handling
```

In the email, include links to your repository and the deployed application — choose either GitHub Pages or Vercel:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

The report must contain:

1. the experiment table from §4 (all 15 rows);
2. the name and version of the AI agent, model, and date of work;
3. your `check.html` result on the deployed page — “Passed: 33 of 33”;

4. the assignment's manual checklist: the result of both items on your final build;
5. summary: what the AI implemented correctly without guidance, and what it missed in each draft; if the agent modified protected files, note which ones.

7. Oral defense

Defense is conducted as an oral discussion of your implementation. You wrote the fixes, so you will modify code live without AI assistance and answer questions while sharing your screen.

Sample questions:

Audit

- Walk through one finding in an AI draft: the `check.html` check that caught it, the root cause, and the change in your `fixes` commit that resolved it.
- Modify code live: add a "Remove note" button on cards using the existing delegated listener, focus the note text after saving, or add a sort `<select>` to the filter form.

Events

- Why does clicking an icon produce a different `event.target`? What does `closest()` return when clicking outside buttons?
- Why does a card created after initial load work with event delegation, but not with listeners bound directly to cards?

Forms and focus

- Why does pressing Enter in a text input reload the page if `submit` is not handled?
- Where does focus land when a focused element is removed from the DOM? How does your code prevent focus loss?

DOM and security

- Why is `textContent` sufficient for notes? What is dangerous about `div.innerHTML = summary; div.textContent`?

AI

- What did the AI miss in each draft? Why, in your opinion?

8. Grading

This lab is worth **3 points** (1 point each for research tasks, practical implementation, and defense), plus **1 bonus point**.

Component	Points	Grading criteria
Research tasks (§4)	1	0.25 points for each subsection §4.1–§4.4: all experiments documented in the table with your prediction and actual result, including explanations for any discrepancies (report item 1)
Practical part (§5)	1	Self-checks: "Passed: 33 of 33" in `check.html` on the deployed site — 0.5; 28 to 32 passed — 0.25. Process: `AI draft` and `fixes` commits following §5.7, manual checklist results (item 4), and AI draft summary (item 5): complete and substantive — 0.5; missing one of the three — 0.25
Defense (§7)	1	Audit questions: explaining an AI draft finding and live code modification without AI — up to 0.5; two questions from other groups in §7 — 0.25 for each complete answer
Bonus point	+1	Either submitted by **19.10.2026** OR completed during class

> **Important!** Without defense, the research and practical components combined are capped at **1 point**: defense confirms that you understand the code you submitted, rather than having merely accepted AI output. The bonus point is awarded even without defense.